number of threads determined at runtime, rather than creating a fixed number as in listing 2.7.

2.4. Choosing the number of threads at runtime

One feature of the C++ Standard Library that helps here is `std::thread::hardware_concurrency()`. This function returns an indication of the number of threads that can truly run concurrently for a given execution of a program. On a multicore system it might be the number of CPU cores, for example. This is only a hint, and the function might return 0 if this information is not available, but it can be a useful guide for splitting a task among threads.

Listing 2.8 shows a simple implementation of a parallel version of `std::accumulate`. It divides the work among the threads, with a minimum number of elements per thread in order to avoid the overhead of too many threads. Note that this implementation assumes that none of the operations will throw an exception, even though exceptions are possible; the `std::thread` constructor will throw if it can't start a new thread of execution, for example. Handling exceptions in such an algorithm is beyond the scope of this simple example and will be covered in chapter 8.

Listing 2.8. A naïve parallel version of `std::accumulate`